题目编辑器 Markdown + LaTeX 支持生产改动报告

生成日期：2026-06-04 | 项目路径：D:\Project\o2iinstitute | 范围：本次编辑器与渲染链路改动

报告摘要：本次改动将题目创建与编辑页面的 `Content HTML` 和 `Solution HTML` 从单纯 HTML textarea 升级为基础编辑器，支持 **Original HTML** 与 **Markdown + LaTeX** 两种输入方式。编辑区始终配套预览，支持全屏左右分栏编辑，并在题目详情页复用同一套安全渲染与 KaTeX 公式渲染逻辑。

一、背景与目标

原系统的题目正文与解析主要依赖直接填写 HTML。这个方式对熟悉 HTML 的管理员可用，但对大量内容录入不够友好，尤其是数学公式、推导过程、表格和普通排版混合出现时，录入成本较高，也容易写出格式不一致的 HTML。

本次目标是在不改数据库 schema、不破坏既有 HTML 内容、不引入复杂富文本编辑器风险的前提下，提供一个轻量但实用的编辑层：

- 题目创建和编辑页面都支持 Markdown + LaTeX 输入。
- 仍然保留 Original HTML 模式，兼容已有内容和高级 HTML 编辑需求。
- 编辑时始终可以看到预览，避免保存后才发现渲染错误。
- Content 和 Solution 使用同一能力，避免解析题目和题干体验割裂。
- 最终落库仍为经过清洗的 HTML，尽量不影响现有发布、详情页、导入与审核链路。

二、功能改动

2.1 编辑模式调整

编辑器只保留两个模式：

模式	用途
Markdown + LaTeX	给日常录题使用。支持 Markdown 文本结构和 LaTeX 公式输入，输入内容会转换为最终提交的 HTML。
Original HTML	给已有 HTML 内容维护、复杂 HTML 微调、导入后修正使用。管理员可以直接编辑最终保存的 HTML。

切换按钮采用类似 segmented switch 的交互，减少“模式很多但不知道选哪个”的问题。

2.2 Content 与 Solution 全量接入

以下位置都已接入新编辑器：

- 创建题目表单中的 `Content HTML`。
- 创建题目表单中的 `Solution HTML`。
- 已有题目编辑区域中的 `Content HTML`。

- 已有题目编辑区域中的 `Solution HTML`。

这样解析题目和题干不再是两套不同输入体验。

2.3 始终存在的预览舞台

无论处于 Original HTML 还是 Markdown + LaTeX 模式，右侧 Preview 都一直显示。预览不是单独的“展示草稿”，而是基于当前最终会提交的 HTML 进行渲染，因此更接近保存后的真实效果。

普通非全屏状态下，左侧编辑区与右侧预览区已经调整为同高、对齐、尽量占满可用空间。左侧 textarea 会随容器高度拉伸，右侧预览也会填满同一高度并在内容过多时滚动。

2.4 全屏编辑

点击 `Expand editor` 后进入全屏弹窗。全屏布局为左侧编辑、右侧预览，右上角提供 `Shrink editor` 收起入口。全屏中仍可以切换 Original HTML 与 Markdown + LaTeX。

这个设计适合长解析、公式推导、多表格内容录入，避免在普通管理页中被表格列宽和页面高度限制。

2.5 Markdown 能力覆盖

当前 Markdown + LaTeX 模式已覆盖以下基础内容：

- 标题：`#`、`##`、`###`。
- 段落、加粗、斜体、inline code。
- 链接：`[OpenAI](https://openai.com)`。
- 图片：`![alt](https://...)`。
- 无序列表和有序列表。
- 引用块：`> quote`。
- Markdown pipe table。
- 行内公式：`$a^2+b^2=c^2$`。
- 块级公式：使用独立 `$$` 行包裹。

2.6 详情页公式渲染

题目详情页中的题干和 Solution 区域改为使用统一的 `RichHtml` 组件。这个组件会先使用现有 sanitizer 输出安全 HTML，然后在客户端查找数学节点，并交给 KaTeX 渲染。

因此，编辑器预览和题目详情页共享同一渲染模型，减少“编辑时能看，详情页不一致”的问题。

三、技术改动细节

3.1 文件级改动

文件	改动说明
`src/components/question-html-editor.tsx`	新增核心编辑器组件。封装模式切换、Markdown 转 HTML、隐藏提交 textarea、预览、全屏弹窗和填满空间布局。
`src/components/rich-html.tsx`	新增统一富文本渲染组件。内部复用 sanitizer，并在客户端执行 KaTeX 渲染。
`src/app/admin/questions/page.tsx`	将创建和编辑表单中的 content_html / solution_html 从普通 Textarea 替换为 QuestionHtmlEditor。
`src/app/questions/[id]/page.tsx`	题目详情页题干和 Solution 改为 RichHtml 渲染，保证详情页也支持公式节点。
`src/lib/sanitize/html.ts`	允许 `div` 和 `span`，用于承载数学节点和表格滚动 wrapper；既有 `data-*` 白名单继续承载 LaTeX 数据。
`src/app/globals.css`	补充 rich-html 图片、表格、数学节点样式。图片自适应，表格横向滚动，公式块可横向滚动。

3.2 数据流

Markdown + LaTeX 模式的数据流如下：

1. 用户在 Markdown textarea 中输入内容。
2. 前端轻量转换器将 Markdown 转为 HTML 字符串。
3. 转换后的 HTML 写入隐藏的 `textarea[name="content_html"]` 或 `textarea[name="solution_html"]`。
4. Preview 使用同一份 HTML 调用 `RichHtml` 渲染。
5. 提交时 server action 读取原有字段名，继续调用 `sanitizeContentHtml` 清洗。
6. 数据库继续保存 HTML，不需要迁移。
7. 详情页读取 HTML 后通过 `RichHtml` 渲染，KaTeX 处理公式节点。

3.3 Markdown 转 HTML 策略

当前没有引入完整 Markdown 解析库，而是做了一个面向题目录入场景的轻量转换器。这样做的原因是第一版需求更偏向可控输入和快速融合，不需要一次性支持所有 Markdown 方言。

- 先对普通文本做 HTML escape，避免用户输入直接形成任意 HTML。
- 再对白名单 Markdown 语法生成受控标签，例如 `a`、`img`、`table`、`blockquote`。
- 链接和图片 URL 只接受 `http` / `https`。
- 表格必须有 divider 行，降低普通竖线文本误判为表格的概率。

- 行内公式转换为 `span data-type="inline-math"`。
- 块级公式转换为 `div data-type="block-math"`。

3.4 安全处理

安全边界分两层：

- 第一层在 Markdown 转换时限制 URL 协议，并对普通文本转义。
- 第二层在最终渲染和保存前继续使用 `sanitize-html` 清洗。

现有 sanitizer 会对链接添加 `target="_blank"` 和 `rel="noopener noreferrer"`，并继续剔除不安全的链接或图片地址。新增的 `div` / `span` 是为了 math 节点和表格 wrapper 服务，不意味着开放脚本执行能力。

3.5 UI 布局优化

本次根据手动测试反馈，额外调整了非全屏和全屏布局：

- 非全屏状态：编辑区和预览区左右对齐，并尽量使用当前页面可用空间。
- 左侧 textarea：使用 flex 布局拉伸高度，避免输入很多内容后只显示很小区域。
- 右侧 Preview：同样填满高度，并在内容过多时内部滚动。
- 表格：外层增加横向滚动 wrapper，避免宽表格破坏页面。
- 图片：设置 block、h-auto、max-w-full，保证不会撑破容器。

四、验证记录

手动验证结果：用户已在 in-app browser 完成测试，并确认功能可以正常运行。

- 已验证 Markdown + LaTeX 模式可输入并渲染。
- 已验证 Original HTML 与 Markdown + LaTeX 的模式切换可用。
- 已验证 Preview 显示正常。
- 已验证 Expand editor 全屏交互可用。
- 已验证非全屏状态下编辑和预览区域需要对齐并用尽空间，该问题已修复。
- 本地执行 `npm run lint` 通过。
- 之前执行 `npm run build` 通过；后续仅布局微调后未再次做浏览器自动测试，符合当时“先不要测试”的要求。

五、上线影响与风险

5.1 兼容性影响

本次没有修改数据库字段，没有改变 server action 接收的字段名，也没有改变已有 HTML 内容的保存格式。因此对现有问题列表、详情页、审核页和导入后的 HTML 内容保持兼容。

5.2 主要风险

风险	说明	缓解方式
Markdown 方言不完整	当前是基础转换器，不覆盖所有 Markdown 语法。	第一版聚焦题目录入常用语法；后续可替换为成熟 Markdown parser。
LaTeX 语法错误	用户输入非法 LaTeX 时，KaTeX 可能显示错误或原文。	KaTeX 使用 `throwOnError: false`，避免页面崩溃。
源文本不可恢复	数据库只保存 HTML，刷新后无法回到原始 Markdown。	作为待改进点单独列出，建议增加 source 字段。
复杂表格能力有限	当前只支持基础 pipe table。	样式层已提供横向滚动；复杂需求可用 Original HTML。

六、待改进点

当前限制：数据库目前只存储转换后的 HTML 内容，不存储用户原始输入的 Markdown + LaTeX 源文本。因此页面刷新或重新进入编辑时，只能回显 HTML，无法看到原来写的 Markdown + LaTeX 文本。

建议后续增加独立源码字段，例如：

- `content_source`：题干源文本。
- `content_source_format`：题干源文本格式，例如 `markdown_latex` 或 `html`。
- `solution_source`：解析源文本。
- `solution_source_format`：解析源文本格式。

这样可以保持发布渲染仍使用 HTML，同时让编辑体验回到用户原始输入格式，便于后续持续维护。

七、结论

本次改动以较小的数据库和后端影响，完成了题目编辑体验的一次关键升级：管理员可以用 Markdown + LaTeX 更自然地录入题目和解析，同时仍保留 Original HTML 的可控性。预览、全屏编辑、详情页 KaTeX 渲染和表格/图片/链接展示修复已经形成一条完整的编辑到展示链路。

当前版本适合作为第一版上线能力。后续最优先的增强方向，是保存 Markdown + LaTeX 源文本，解决刷新后无法回到原始编辑文本的问题。